

Week 2 Monday

Variables, Expressions, and Types

CS1

Essential Question

How does a program store a value, compute something from it, and show the result predictably?

Deitel & Deitel, Intro to Python for Computer Science and Data Science, Chapter 2, "Introduction to Python Programming," §§2.1–2.4, 2.6, 2.8

Variables and Assignment

- A variable stores a value for later use; `=` is assignment, not algebraic equality.
- The right side evaluates completely first, then binds to the name on the left.
- `total = x + y` reads “total is assigned the value of `x + y`.”

`x = 7`

`y = 3`

`total = x + y    # 10`

Arithmetic and Precedence

- `+` `-` `*` `**` as expected; `/` is true division (always float); `//` is floor division (truncates toward $-\infty$); `%` is remainder.
- Precedence: parentheses, then `**` (right-to-left), then `*` `/` `//` `%` (left-to-right), then `+` `-`.
- Parentheses change results, not just readability.

`7 / 4` `# 1.75`

`-13 // 4` `# -4`

`10 * (5 + 3)` `# 80`

`7 // 4` `# 1`

`17 % 5` `# 2`

`10 * 5 + 3` `# 53`

Formatted Output

- `print` joins a comma-separated list with spaces.
- f-strings (`f"..."`) embed expressions directly in `{}`, with an optional format spec after `:`.
- Use f-strings for anything with more than one value in the line.

```
print('Sum is', 7 + 3)           # Sum is 10
price = 19.999
print(f"Price: ${price:.2f}")    # Price: $20.00
```

Input Always Returns a String

- `input()` returns a `str`, even when the user types digits.
- `'7' + '3'` concatenates to `'73'` – not addition.
- Convert first: `int()` or `float()`, then compute.

```
value1 = input('Enter first number: ')    # '7'
value2 = input('Enter second number: ')   # '3'
value1 + value2                           # '73' (concatenation!)
int(value1) + int(value2)                 # 10
```

Objects, Types, and Dynamic Typing

- Every value is an object; every object has a type.
- Python is dynamically typed: a variable can be rebound to a value of a *different* type at any time.
- `bool` is a subtype of `int` – `True + True` is 2.

```
type(7)          # <class 'int'>
x = 7
x = 'dog'        # legal -- x now refers to a str
True + True     # 2
```

Common Type Errors

<code>TypeError</code>	mixing incompatible types, e.g. <code>'3' + 3</code>
<code>ValueError</code>	<code>int()/float()</code> on a non-numeric string
<code>ZeroDivisionError</code>	<code>/</code> or <code>//</code> with a zero denominator
<code>NameError</code>	using a variable before it's assigned
<code>SyntaxError</code>	<code>=</code> where <code>==</code> was meant, or a stray character

Worked Example: This Week's Gate

Frontier Settlement genre, from
assignments/odyssey_gates/week-02.md:

```
rations_per_colonist = 2
colonist_count = 40
food_stock = rations_per_colonist * colonist_count
print(f"The settlement begins with {food_stock} days of food.")
```

Same shape works for all four genres: a variable, an expression, a formatted sentence.

Runnable Example File

- `lessons/code/week02_variables_expressions_types.py` – every example from today, runnable with no input required.
- Includes the historical caffeine half-life lab (fall 2021 onward) as a working function, using this lecture's exponentiation operator and f-string formatting.
- `https://github.com/jeremy-evert/computer_science_1/blob/main/lessons/code/week02_variables_expressions_types.py`

Closing Takeaway

A variable stores a value; an expression computes a new one from it; formatted output shows the result predictably. Everything in this week's gate is that pattern, applied to your world.

- Deitel & Deitel, *Intro to Python for Computer Science and Data Science* (Pearson, 2020), Chapter 2, §§2.1–2.4, 2.6, 2.8, pp. 49–61, 66–68.
- This deck covers Deitel §§2.1–2.4, 2.6, 2.8 only – §2.5 (triple-quoted strings) and §2.7 (the `if` statement) are deferred to Week 3.

Python Documentation Cited Today I

- Identifiers and keywords: https://docs.python.org/3/reference/lexical_analysis.html#identifiers
- Operator precedence: <https://docs.python.org/3/reference/expressions.html#operator-precedence>
- Formatted string literals (f-strings): <https://docs.python.org/3/tutorial/inputoutput.html#formatted-string-literals>
- Numeric types (int, float, complex): <https://docs.python.org/3/library/stdtypes.html#numeric-types-int-float-complex>
- Boolean type: <https://docs.python.org/3/library/stdtypes.html#boolean-type-bool>
- Built-in int/float/str:
<https://docs.python.org/3/library/functions.html#int>,
<https://docs.python.org/3/library/functions.html#float>,
<https://docs.python.org/3/library/functions.html#func-str>
- PEP 8 style guide: <https://peps.python.org/pep-0008/>